

Hardware-Constrained Architecture Search for Lane-Change Intention Prediction from Vehicle Time Series

Sepehr Mohammady, [TODO: co-authors, order], [TODO: supervisor]

Abstract—Predicting a driver’s lane-change intention early enough to act on it is a time-series classification problem with hard deployment limits: the model has to run on automotive-grade microcontrollers within tight memory and latency budgets. We apply constrained neural architecture search to the Lane Change Intention Recognition driving dataset [3] (windows of 50 samples over 31 signals) and obtain compact one-dimensional convolutional networks for three tasks: three-class intention classification and time-to-lane-change regression for left and right maneuvers. On an STM32H7B3I-DK the searched classifier reaches 92.1% test accuracy with 84k parameters in 3.63 ms, 335 KB of flash and 9.2 KB of RAM, and an 8k operating point holds 91.3% at 0.79 ms in 37 KB; the regression models reach 0.45 s (right) and 0.47 s (left) RMSE, improving on the published baseline (Transformer, 0.51 s, deployed in FP32 [1]). Against an internal 441k reference CNN our classifier is $5.2\times$ smaller in flash and $4\times$ smaller in RAM at higher accuracy. All footprint and latency figures are measured on the target hardware rather than estimated, in FP32 as the baseline deploys, with int8 and quantization-aware int8 reported as additional operating points.

Index Terms—Neural architecture search, lane-change intention, time-series classification, TinyML, embedded inference, STM32.

I. INTRODUCTION

[TODO: Motivation: intention prediction as advance warning for ADAS; why on-device inference (latency, privacy, cost, no connectivity assumptions); the gap: published DMIR models target accuracy, not deployment budgets.]

Contributions:

- 1) a constrained search space and objective tuned to one-dimensional driving signals under microcontroller budgets;
- 2) models that improve on the published TTLC baseline (RMSE 0.51 s) and on an internal 441k reference classifier, at a fraction of the deployed footprint;
- 3) measured deployment numbers (flash, RAM, latency) on the STM32H7B3I-DK (Cortex-M7) and NUCLEO-F401RE (Cortex-M4), with the full pipeline released as open source.

The authors are with the ELIOS Lab, Department of Electrical, Electronic and Telecommunications Engineering and Naval Architecture (DITEN), University of Genoa, Genoa, Italy.

This work was supported by the SYNERGIES project [TODO: grant number and official acknowledgment text].

II. RELATED WORK

A. Lane-change intention prediction

[TODO: The LC dataset and framework baseline [1] (TTLC regression, Transformer RMSE 0.51 s, FP32 deployment on STM32H7B3I/F401); earlier HighD-based TTLC work (RMSE 0.63–0.7); MicroNAS for time series and TinyTNAS as closest NAS relatives.]

B. Constrained architecture search for microcontrollers

[TODO: muNAS [2], MCUNet/TinyNAS, MicroNets; what transfers to 1D time series and what does not.]

III. DATA

We use the Lane Change Intention Recognition dataset [3], [4] (50 human drivers in the CARLA simulator, 10 Hz, over 3,400 annotated lane changes; MIT license), prepared as fixed windows of $T=50$ samples (5 s) over $F=31$ channels. The official feature list counts 30; we verified that the extra channel is not a timestamp but a binary `car2Present` flag, so all 31 are retained. The feature set includes the driver’s turn-signal state as two binary channels, which we ablate below. Classification uses balanced splits of 94,620 / 16,332 / 19,722 windows (train/val/test) over three classes: no intention, right lane change, left lane change. The regression tasks map a window to the time until the lane change, $t \in [0, 4]$ s discretized at 0.1 s, with 39,313 / 7,742 / 8,097 (LCL) and 33,201 / 5,828 / 7,073 (LCR) windows.

The splits are user-independent. We spot-checked five drivers (test users 2 and 13, validation users 5 and 10, training user 22) by per-window Pearson correlation ($r > 0.9999$) against the raw session logs, and found each only in its assigned split, consistent with the driver-wise split released with [1]. The test splits contain isolated spikes (up to 5×10^6) on the right/left `curvatureDx` channel pair, a numerical-derivative artefact absent from training; we clip validation and test features to the per-channel training range and leave training data untouched.

IV. METHOD

A. Search space

The search space is built from up to ten convolutional blocks of at most three layers each, every layer being a standard, depthwise or 1×1 one-dimensional convolution with kernel size in $\{3, 5, 7\}$ and 1–128 filters, each with optional stride-2,

pre-pooling, batch normalisation and ReLU. A block is placed serially or as a parallel branch merged with its predecessor, and the head is a global average or max pool with size in $\{2, 4, 6\}$ followed by one to three fully-connected layers of 10–256 units. The operator set is restricted to layers the ST Edge AI compiler maps efficiently.

B. Constraints and search algorithm

We use aging evolution: a population of architectures is sampled, the fittest of a random subset is mutated by a network morphism, its child is trained, and the oldest member is retired. Fitness combines four objectives—validation error, peak RAM, model size and multiply–accumulate count—by the random scalarisation of [2]: each is normalised by a user budget and divided by a random weight resampled every round, so a candidate exceeding any budget is penalised first. Budgets were set from the STM32H7B3I-DK, the deployment platform of the published baseline, so the on-device comparison is like for like. Each task ran 150 rounds with a population of 50 and a sample size of 15; all four searches completed overnight on a single laptop GPU (RTX 5070), against thousands of GPU-days for early reinforcement-learning and evolutionary NAS [5], [6]. This is a different-task comparison offered as context only, but it follows from searching directly in the microcontroller-scale regime, where every candidate trains in minutes.

C. Discovered architecture

Across tasks the search independently arrives at the same microcontroller-friendly recipe: 1×1 convolutions to re-shape the channel dimension, depthwise convolutions for temporal structure, aggressive early downsampling, and a small flattened head (98–308 features against the reference’s 6400). Topology, however, follows the task: the classifiers are essentially sequential, whereas the regression models fan out into two (LCR) and three (LCL) parallel branches merged by additions. The search space permits branches everywhere; only regression exploited them. The classification model reduces the sequence $50 \rightarrow 25 \rightarrow 13 \rightarrow 7 \rightarrow 4$ before its head:

$50 \times 31 \rightarrow \text{MaxPool} \rightarrow \text{DWConv}(s2) \rightarrow \text{Conv}_{1 \times 1}(77) \rightarrow \text{DWConv} \rightarrow \text{DWConv}(s2) \rightarrow \text{Conv}_{1 \times 1} \rightarrow \text{Add} \rightarrow \text{AvgPool} \rightarrow \text{FC}(223) \rightarrow \text{FC}(23) \rightarrow \text{FC}(3)$

This explains the footprint gap quantitatively. Both our model and the reference CNN spend most of their flash in the fully-connected head, so the gain does not come from avoiding a large head; it comes from what enters it. The reference holds full temporal resolution and flattens a 50×128 map into 6400 features, giving a 64×6400 layer of 1.64 MB (93% of its weights). Ours pools first and flattens only $4 \times 77 = 308$ features, so it affords a *wider* head (223 units) in a ~ 276 KB layer, a $5.9\times$ reduction. The same downsampling cuts the convolutional work, yielding the $12.4\times$ fewer MACs and the $9.2\times$ measured speed-up of Table II.

TABLE I
TEST-SET COMPARISON. THE INTERNAL REFERENCE CNN (UNPUBLISHED) WAS EVALUATED ON OUR TEST SET; THE TRANSFORMER RMSE VALUES ARE AS REPORTED BY THE AUTHORS AND COULD NOT BE RE-RUN, THEIR CUSTOM ENCODER LAYER NOT BEING AVAILABLE. SIZE IS THE PARAMETER COUNT.

Model	Params	Acc. (%)	LCR	LCL
Reference CNN (cls)	441 k	91.7	—	—
Reference Transf. (reg, RMSE)	333/49 k	—	0.42	0.44
Our DSCNN baseline	10.5 k	91.5	0.32 [†]	0.33 [†]
Ours searched (cls / reg MAE)	84 k [‡]	92.1	0.29 [†]	0.32 [†]
Ours compact	8 k	91.3	—	—
Ours, no turn-signal	21 k	91.1	—	—

[†]MAE. The LCR model comes from an MAE-objective search and the LCL model from an RMSE-objective re-run; reference regression is reported as RMSE. Our LCR RMSE is 0.45 and LCL 0.47. [‡]Parameter counts differ per task: 84 k (classification), 117 k (LCR), 106 k (LCL).

V. EXPERIMENTS

A. Setup

Each candidate is trained for 100 epochs on the regression tasks and 80 on the classification tasks. Searches ran under WSL2 on a single RTX 5070 laptop GPU (TensorFlow 2.21), in fresh-process chunks resuming from the aging-evolution checkpoint, which bounds the memory growth of a long-lived search process. Each task was searched once rather than repeated across seeds, so we report the selected architectures rather than a variance over runs; every candidate and every reported evaluation is recorded in the released `experiments.jsonl`.

B. Results

Table I reports test-set metrics. All searched-model numbers are our own test-set evaluation of the saved models, from searches run to their full 150-round budget; measured footprint and latency follow in Table II.

For classification the searched model is both smaller and more accurate than the 441 k reference CNN (92.1% at 84 k parameters; an 8 k model still reaches 91.3%). Regression improves on the published single-TTLC RMSE of 0.51 [1] (ours 0.45 LCR, 0.47 LCL) at a comparable parameter count (117 k and 106 k against ~ 54 k), so the regression result is an accuracy and deployability claim, not a size one; the comparison is also indicative rather than strict, since the baseline reports a single combined-direction TTLC model on 30 channels where we train one model per direction on 31. We do not claim to beat the internal-reference Transformers on regression RMSE (0.42/0.44), which remain competitive small models; an RMSE-aware search narrowed but did not close that gap. Removing the turn-signal channels drops classification accuracy by about one point (92.1% to 91.1%), evidence that the model anticipates from vehicle dynamics and surrounding traffic rather than reading a declared signal. Those two channels alone reach 81.5% accuracy, so the ablation measures how much the model relies on a signal a driver may never set.

TABLE II
MEASURED ON-DEVICE FOOTPRINT AND LATENCY, STM32H7B3I-DK
(CORTX-M7, 280 MHz), ST EDGE AI CORE 4.0.1, FP32; ACCURACY IS
OUR HOST TEST-SET EVALUATION OF THE SAME DEPLOYED FILES.

	Reference CNN	Ours	Ours (tiny)
Parameters	441 k	84 k	8 k
Accuracy (%)	91.7	92.1	91.3
Latency (ms)	33.52	3.63	0.79
Flash (KB)	1728	335	37
RAM (KB)	38.3	9.2	9.2
MACC (M)	1.97	0.16	0.03

C. Deployment on STM32

Target platform: STM32H7B3I-DK discovery kit (Cortex-M7 at 280 MHz, 1184 KB internal SRAM, 2 MB flash), the same MCU family used by the baseline framework [1], which deployed FP32 models only. We therefore compare in FP32, like for like, benchmarking both models on the same board through the ST Edge AI Developer Cloud (Core 4.0.1, balanced optimization). Table II reports measured, not estimated, figures.

The searched model is $9.2\times$ faster and $5.2\times$ smaller than the reference CNN while being more accurate; a second operating point on the same front gives up 0.4 points of accuracy for a $42.3\times$ speed-up and a $46.6\times$ smaller model that infers in under a millisecond.

The smaller operating point also changes what hardware is reachable, and here the difference is categorical rather than incremental. On a NUCLEO-F401RE (Cortex-M4, 84 MHz, 512 KB flash) the tiny classifier runs in 4.376 ms and occupies 7.2% of flash, while the reference CNN does not fit at all: its 1728 KB of flash exceed the board’s entire 512 KB by $3.4\times$, which no compiler setting can recover. Nor is the small model the only one that reaches this board: the full classifier runs there in 18.35 ms at 65.5% of flash, and its quantized form in 7.381 ms at 25.0%, so the accuracy headline itself is available on the Cortex-M4, as is the LCL regression model at 162.5 ms. The searched models therefore open a device class the reference is locked out of, consistent with the baseline’s own report that its Transformer did not fit this board. One build does not follow: the widest regressor in float32 occupies 90.5% of flash and returned no measured time, while every build that ran leaves at least 98 KB free. Quantizing that same network to int8—identical architecture, operators and board, 28.7% of flash—makes it run in 28.10 ms, which isolates the cause: the binding constraint is flash *headroom* for the runtime, not model size or operator support. Size arithmetic alone would have called the float32 build deployable. On this board quantization is a speed and memory optimisation, and it is also what makes the model run at all.

Operation counts are a poor latency budget at this scale, at least on ST’s float32 kernels. The tiny model costs 7.0 cycles per MAC on the M7 and 11.6 on the M4, on cores with single-cycle MAC instructions, so inference is bound by per-op overhead and memory traffic rather than arithmetic, and per-layer time does not follow per-layer MACs. A zero-MAC pooling operator is among the most expensive layers in

the tiny classifier, and in the full one the slowest layer is a depthwise convolution carrying 0.8% of the model’s MACs. The int8 build inverts this, with per-layer time tracking MACs. MAC count therefore remains a good predictor of whole-model latency across models, but a poor one within a model.

The per-layer breakdown explains why. Both our classifier and the reference spend most of their flash in the fully-connected head, so the gain is not from avoiding a large head but from what enters it: the reference flattens a 50×128 map into 6400 features, giving a 64×6400 layer of 1.64 MB (93% of its weights), whereas the searched models pool first and flatten only 308 (or 98) features, cutting that layer $5.9\times$.

Across the searched models this reliance falls monotonically with how much of the search space the task exploits. The largest fully-connected layer holds 93% of weights in the reference, 90.9% in our tiny classifier and 82.5% in the full one, but 39.6% in the LCR model and only 11.4% in the LCL model, whose largest layer is a convolution. The same ordering appears in topology, as noted above: the classifiers are sequential while the regression models fan out into two and three branches. Topology follows the task, discovered rather than imposed.

RAM behaves differently and is worth stating plainly, because three distinct regimes appear across our models. Both searched classifiers measure ~ 9.2 KB regardless of a $10.5\times$ parameter gap: the 50×31 float32 input tensor alone occupies 6.2 KB and no architecture can go below it in FP32, so they are *input-bound*. The LCR model measures 20.3 KB and is *width-bound*: its widest branch emits a 25×116 tensor of 11.6 KB, and the peak working set is that plus the input. The LCL model measures 27.6 KB and is neither; it is *liveness-bound*. Its widest activation (13.0 KB) and the next-widest (12.0 KB) have disjoint live ranges and are never co-resident, so no width-based rule reaches the measurement. Instead its three-way merge pins the 6.2 KB input and two 3.3 KB branch outputs while the 12.0 KB output of the branch then executing is live, giving a 24.8 KB peak; the remaining ~ 2 KB is not derivable from the graph and reflects allocator offset assignment and alignment. The general statement is therefore that peak RAM is the maximum over the schedule of all simultaneously live tensors, which collapses to $\max(\text{input}, \text{widest in} + \text{out})$ only for a chain. This is why the RAM advantage ($\sim 4\times$) is far smaller than the flash advantage.

The input-bound regime is not a property of the network but of the tensor interface, and we verified this by changing only the interface. Exporting the int8 classifier with int8 rather than float32 input and output tensors—the weights, graph and accuracy are untouched, 86.86% either way—removes the 6.2 KB float32 input buffer and both cast operators. Measured on the same board, RAM falls from 8,096 to 5,444 B and inference from 1.885 to 1.752 ms, because casting 1,550 elements every inference is work the board no longer does. The memory reduction is smaller than the 4.65 KB of buffer removed, which is the same maximum-over-live-tensors point again: shrinking the input does not subtract from the arena, it merely stops the input being the binding constraint, after which the widest internal activation sets the floor at 3.5 KB. Reporting int8 with a float32 interface therefore understates

both its memory advantage— $1.17\times$ against float32 before the change, $1.74\times$ after—and its speed, and we note it because the default converter settings produce exactly that misleading configuration.

On quantization, the ST toolchain accepts only float32/int8/uint8 tensors [7]. Full post-training int8 shrinks the classifier to 104 KB and 1.885 ms but costs 5.2 accuracy points on these wide-dynamic-range inputs ($92.1\% \rightarrow 86.9\%$; LCR MAE $0.287 \rightarrow 0.449$), so we report float32 as the headline configuration, which also matches the baseline’s own FP32 deployment. Quantization-aware fine-tuning recovers most of that int8 loss: the classifier returns to 89.8% (from 86.9% post-training), closing 57% of the gap to float32. Combined with the int8 interface described above, it gives the fastest configuration we measured: 89.9% at 1.435 ms in 128 KB of flash and 6.2 KB of RAM, which is $2.5\times$ the speed, $2.6\times$ less flash and $1.5\times$ less RAM of the float32 build for 2.2 accuracy points. Deployment therefore offers three defensible operating points rather than one: float32 for accuracy and like-for-like comparison with the FP32 baseline, QAT int8 for speed, and post-training int8 for the smallest footprint (104 KB, 5.3 KB RAM). The QAT tooling registers only 2D layers, so for that configuration the 1D graph is re-expressed with width-1 kernels; the computation is identical, the accuracy is read through the int8 interpreter, and the post-training int8 models deploy the 1D graph natively. The same procedure does *not* transfer to the regression heads: it recovers only 11% of the gap for LCR and degrades LCL outright ($0.344 \rightarrow 0.362$ s MAE), a result stable across fine-tuning rates. We read this as a task asymmetry rather than a tuning failure: classification needs only the argmax of three logits, which tolerates coarse activations, while a continuous time-to-lane-change estimate does not, and int8 costs the classifier 5.7% of its accuracy against a 57% increase in LCR error. Weight fine-tuning cannot restore information lost in the activation representation, so we report float32 as the only accurate regression configuration and int8 there purely as a deployability fallback. An int16 $\times$ 8 scheme (int8 weights, int16 activations) preserves accuracy almost exactly offline ($92.1\% \rightarrow 92.2\%$; LCR MAE $0.287 \rightarrow 0.286$), but the toolchain does not deploy it [7]. We verified this on hardware rather than trusting the documentation: a 117.9 KB int16 $\times$ 8 file returns 326.15 KB of weights, byte-identical to the float32 build, while a genuinely int8 control compresses to 83.28 KB. The scheme is silently dequantized to float32, so we report it as an offline bound only. The episode is worth one sentence of method: a toolchain that accepts an unsupported scheme and quietly widens it will produce a benchmark that looks like a result.

VI. CONCLUSION

[TODO: two or three sentences; no new claims.]

REFERENCES

- [1] L. Forneris *et al.*, “A deployment-oriented simulation framework for deep learning-based lane change prediction,” *IEEE Signal Process. Lett.*, vol. 33, pp. 136–140, 2026, doi: 10.1109/LSP.2025.3638676.
- [2] E. Liberis, Ł. Dudziak, and N. D. Lane, “ μ NAS: Constrained neural architecture search for microcontrollers,” in *Proc. 1st Workshop Mach. Learn. Syst. (EuroMLSys)*, 2021, pp. 70–79.
- [3] L. Forneris *et al.*, “Lane change intention recognition dataset,” Zenodo, 2025, doi: 10.5281/zenodo.16686054.
- [4] L. Forneris *et al.*, “Setting up a lightweight simulation environment for automated driving dataset collection,” in *Applications in Electronics Permeating Industry, Environment and Society (ApplePies)*, Springer LNEE, 2024, pp. 156–163, doi: 10.1007/978-3-031-84100-2_19.
- [5] B. Zoph and Q. V. Le, “Neural architecture search with reinforcement learning,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.
- [6] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, “Regularized evolution for image classifier architecture search,” in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4780–4789.
- [7] STMicroelectronics, “ST Edge AI Core Technology 4.0.0 — Quantization; Supported operators (TFLite),” online documentation, stedgeai-dc.st.com/assets/embedded-docs/, accessed 2026-07-14.